

Project - High Level Design

on

Multi-Agent Manufacturing Creator

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Bhaskar Sharma	EN22CS301261
2.	Chitrak Jiyal	EN22CS301303
3.	Darshana Matade	EN22CS301309
4.	Dipanshi Yadav	EN22CS301337

Group Name: 08D3

Project Number: AAI-32

Industry Mentor Name: Mr.Ashwin

University Mentor Name: Prof. Maya Yadav Baniya

Academic Year: 2025-26

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1. Security Aspects
 - 7.2. Performance Aspects
8. References

1. Introduction

1.1 Scope of the Document

This document describes the high-level architecture, workflow, and system design of the Multi-Agent Manufacturing System, an Agentic AI-based application designed to automate supplier sourcing and report generation using collaborative AI agents.

1.2 Intended Audience

- System Architects: To understand the multi-agent orchestration and modular system design.
- Developers: To review integration of Crew AI, LangChain, and vector databases.
- AI Engineers: To understand agent specialization and communication workflows.

1.3 System Overview

The Multi-Agent Manufacturing System is an intelligent collaborative agent framework that automates supplier sourcing tasks. It leverages specialized agents:

- Researcher Agent → extracts relevant supplier data
- Writer Agent → structures and formats the final output
- Orchestrator → manages workflow and communication

The system integrates LLMs (Gemini), vector databases, and agent orchestration (Crew AI) to transform raw manufacturing datasets into meaningful insights and structured reports.

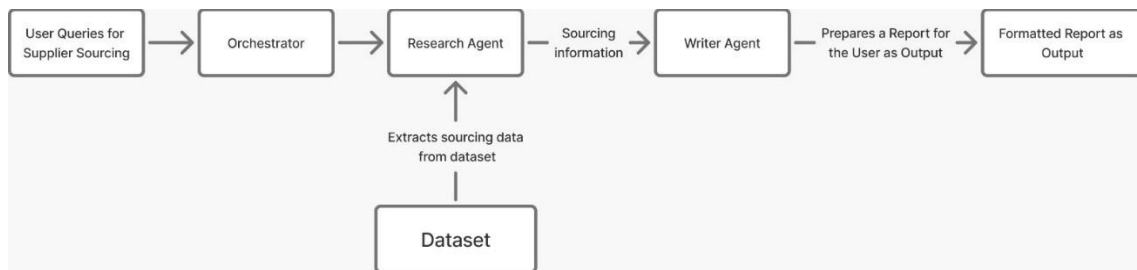
2. System Design

2.1 Application Design

The application follows a multi-agent modular architecture built using:

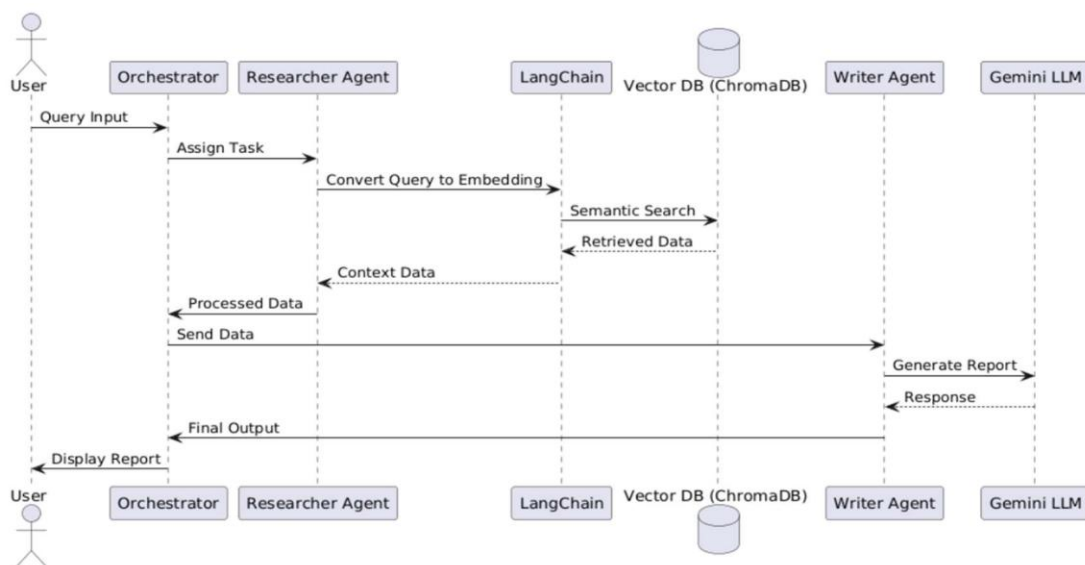
- Crew AI for agent orchestration
- LangChain for embeddings and retrieval
- ChromaDB for vector storage
- Streamlit for user interface
- Python as the core backend

The design follows an Orchestrator-Agent pattern, where the orchestrator controls task delegation and execution flow between agents.



2.2 Process Flow

1. User Input: User provides a supplier sourcing query
2. Orchestrator Activation: Initializes workflow and context
3. Research Phase:
 - Task assigned to Researcher Agent
 - Queries vector database for relevant data
4. Data Transfer: Results passed back to Orchestrator
5. Writing Phase:
 - Writer Agent formats structured response
6. Final Output: Report delivered to user via UI



2.3 Components Design

- **Orchestrator:**
Controls workflow, manages agent interactions, and maintains system state
- **Researcher Agent:**
Retrieves supplier and manufacturing data from vector database
- **Writer Agent:**
Converts extracted data into structured reports
- **Vector Database (ChromaDB):**
Stores embeddings for efficient semantic retrieval
- **User Interface (Streamlit):**
Provides interactive input/output interface

2.5 Key Design Considerations

- **Agent Specialization:**
Separation of concerns improves accuracy and efficiency
- **Scalability:**
Modular agents allow easy addition of new capabilities
- **State Management:**
Orchestrator ensures consistent workflow execution
- **Data Retrieval Efficiency:**
Vector database enables fast and relevant search
- **Inter-Agent Communication:**
Structured hand-off protocols ensure reliable data flow

2.6 API Catalogue

Provider	Model	Purpose
Crew AI	Agent Framework	Task orchestration and agent management
Google Gemini	LLM	Reasoning, summarization, and response generation
LangChain	Embedding	Data processing and retrieval pipeline

ChromaDB	Vector Database	Storage and semantic search
Streamlit	UI Framework	User interaction and visualization

3. Data Design

3.1 Data Model

The system operates on a retrieval-based structured data model:

JSON

```
{  
  "query": "string",  
  "supplier_data": ["list"],  
  "processed_info": "string",  
  "final_report": "string"  
}
```

3.2 Data Access Mechanism

- Data is stored in ChromaDB vector database
- Retrieval is done via semantic search using embeddings
- APIs interact with Gemini for reasoning and formatting

3.3 Data Retention Policies

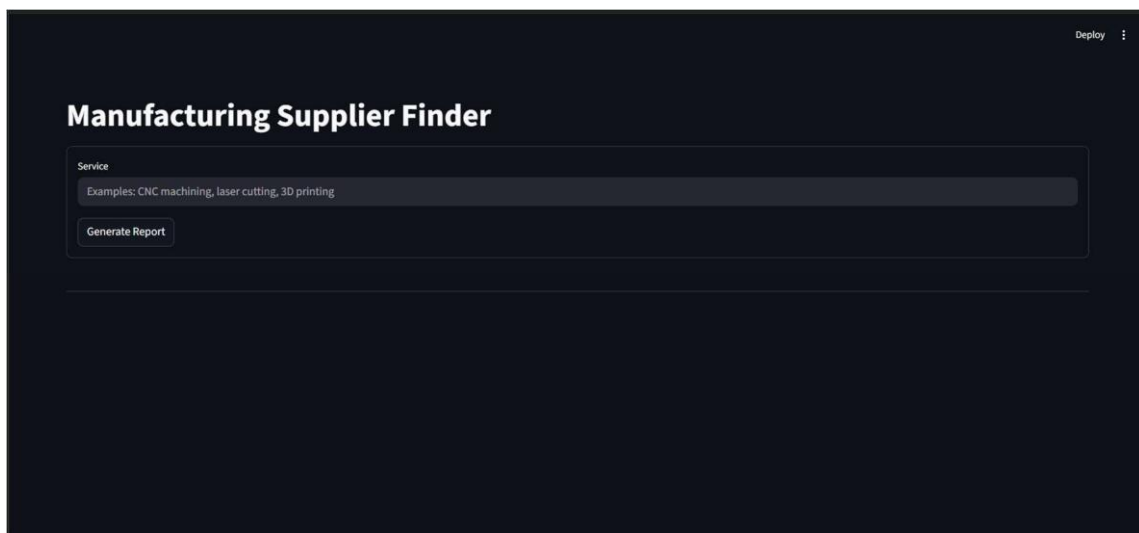
Currently, the application uses Volatile Retention. Data exists only within the `st.session_state` and is cleared upon browser refresh or session termination.

3.4 Data Migration

N/A (Stateless application).

4. Interfaces

- GUI: Streamlit-based web interface.
- External APIs:
 - Gemini API (LLM processing)
 - Crew AI (agent orchestration)
- Database Interface: ChromaDB for vector storage



5. State and Session Management

- Managed by Orchestrator
- Maintains workflow state across agents
- Streamlit session ensures persistence during execution

6. Caching

The application does not currently implement `@st.cache_data`. However, the persistence of the result in session state acts as a manual cache for the duration of the user's visit.

7. Non-Functional Requirements

7.1 Security Aspects

- Token Management: API keys are retrieved via `os.environ.get`, ensuring no secrets are hardcoded.
- Input Sanitization: Uses `unsafe_allow_html` cautiously for UI styling, though user inputs are primarily handled as text data for the models.

7.2 Performance Aspects

- Fast retrieval using embeddings
- Dependent on LLM response time (Gemini)
- Efficient multi-agent execution via orchestration

8. References

- [Streamlit Documentation](#)
- [Google Gemini API Guide](#)
- [Crew AI Documentation](#)